

Worker vs Shared Worker

First, Worker

The Worker interface of the Web Workers API represents a background task that can be created via script, which can send messages back to its creator.

- Creating a worker is done by calling the `Worker("path/to/worker/script")` constructor.
- Workers may themselves spawn new workers, if those workers are hosted at the same origin as the parent page.
- Note that not all interfaces and functions are available to web workers.

For example, suppose I have two tasks to do, one of which is to sum all values from 1 to N

And the other task is to create a button to change the background color of the page. Here the worker is helpful as it simulates asynchronous thread or a separate thread that calculates the Summation separately and changing the background will be executed quickly as it is much faster than doing calculations.

A **Web Worker** runs a script in a **separate thread**, so heavy computations don't block the main thread.

- **Main thread:** Handles UI updates, clicks, rendering.
- **Worker thread:** Handles heavy computation like summing numbers.

This way:

- The page stays **responsive**.
- You can click the button to change the background even if the sum is still being calculated.

The main thread code (main.js): Handles UI updates:

```
1  // Create a worker
2  const worker = new Worker('worker.js');
3
4  // Send N to the worker
5  worker.postMessage(1000000000);
6
7  // Receive result from worker
8  worker.onmessage = (e) => {
9      console.log('Sum =', e.data);
10 };
11
12 // Button to change background
13 document.getElementById('bgButton').addEventListener('click', () => {
14     document.body.style.backgroundColor = 'lightblue';
15 });
16
17
```

The Worker-thread code to calculate summation from 1 to 1000000.

```
1  // Listen for message from main thread
2  onmessage = function(e) {
3      const N = e.data; // a million 1 -> million
4      let sum = 0;
5      for (let i = 1; i <= N; i++) {
6          sum += i;
7      }
8      // Send result back to main thread
9      postMessage(sum);
10 };
11
```

Second, Shared Worker.

The Shared Worker interface represents a specific kind of worker that can be accessed from several browsing contexts, such as several windows, iframes or even workers. They implement an interface different than dedicated workers and have a different global scope, `SharedWorkerGlobalScope`.

- A **Shared Worker** can be accessed by **multiple scripts or browser tabs/windows** from the **same origin**.
- Unlike a dedicated worker, which is private to one script, a shared worker can **share data or computation** across multiple pages or tabs.
- Communication happens via **ports** (`MessagePort`).

Use case:

Suppose multiple tabs of your app need the **same computation or data** — instead of each tab doing it separately, they all talk to one shared worker.

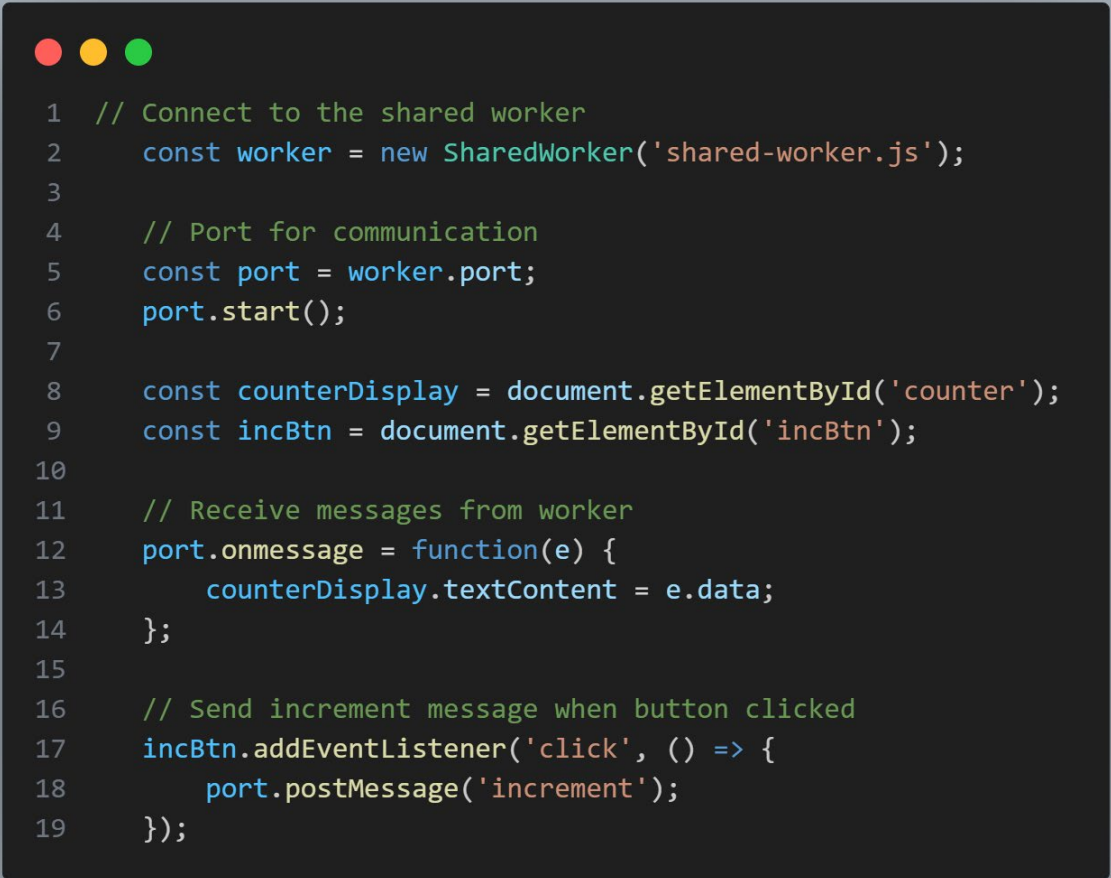
Example: Shared Counter Across Tabs

Purpose:

- Keep a counter that increments whenever **any tab** clicks a button.
- All tabs show the **same counter** in real-time.

Here is the first Thread:

The first Thread: A page has the incremental button which increases the counter by one.

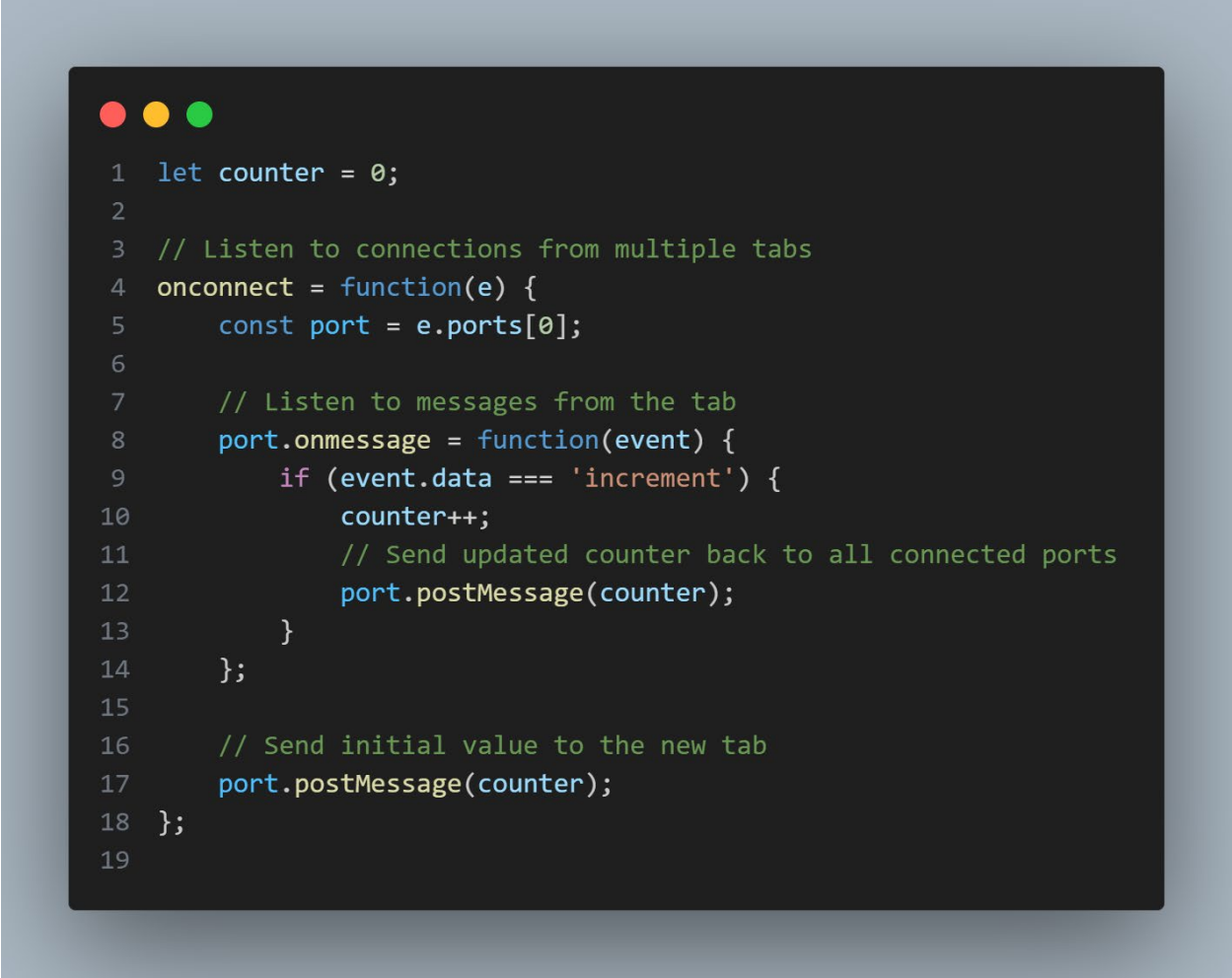


```
1  // Connect to the shared worker
2    const worker = new SharedWorker('shared-worker.js');
3
4    // Port for communication
5    const port = worker.port;
6    port.start();
7
8    const counterDisplay = document.getElementById('counter');
9    const incBtn = document.getElementById('incBtn');
10
11   // Receive messages from worker
12   port.onmessage = function(e) {
13       counterDisplay.textContent = e.data;
14   };
15
16   // Send increment message when button clicked
17   incBtn.addEventListener('click', () => {
18       port.postMessage('increment');
19   });
```

Here is the Secondary Thread :

The second Thread: A page has the same incremental button as the first thread.

Hence, the two pages or tabs have the same shared worker.



```
1  let counter = 0;
2
3  // Listen to connections from multiple tabs
4  onconnect = function(e) {
5      const port = e.ports[0];
6
7      // Listen to messages from the tab
8      port.onmessage = function(event) {
9          if (event.data === 'increment') {
10             counter++;
11             // Send updated counter back to all connected ports
12             port.postMessage(counter);
13         }
14     };
15
16     // Send initial value to the new tab
17     port.postMessage(counter);
18 };
19
```

End of Research